Arytmetyka liczb binarnych

Przypomnienie pojęć podstawowych

System pozycyjny: wartość każdej cyfry zależy od jej pozycji w zapisie.

Podstawa (base) systemu: określa liczbę różnych cyfr (np. 2 w systemie binarnym, 10 w dziesiętnym).

Zapis ogólny liczby w systemie o podstawie B:

$$N = a_n \times B^n + a_{n-1} \times B^{n-1} + \cdots + a_1 \times B^1 + a_0 \times B^0$$

Każda cyfra a_i jest z zakresu [0,B−1].

System dziesiętny (decymalny) – krótkie przypomnienie

Podstawa: 10, cyfry 0–9.

Przykład: 439_{10} to:

$$4 \times 10^2 + 3 \times 10^1 + 9 \times 10^0 = 400 + 30 + 9 = 439.$$

Najczęściej używany w życiu codziennym (system „naturalny” dla ludzi).

System binarny (dwójkowy) – wprowadzenie

- **Podstawa**: 2, cyfry: 0 i 1.
- **Zastosowanie**: układy cyfrowe, komputery (stan *wysoki* = 1, *niski* = 0).
- **Przykład**:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

$$= 8 + 0 + 2 + 1 = 11_{10}.$$

Konwersja z dziesiętnego na binarny – metoda dzielenia przez 2

Podziel liczbę przez 2, zapisz resztę (0 lub 1).
Weź całkowity wynik dzielenia i powtórz procedurę, aż otrzymasz 0.
Odczytaj reszty od końca (od ostatniej do pierwszej) – to wynik w systemie binarnym.
Przykład: 13 na binarny:

$13 \div 2 = 6$, reszta $= 1$
$6 \div 2 = 3$, reszta $= 0$
$3 \div 2 = 1$, reszta $= 1$
$1 \div 2 = 0$, reszta $= 1$

Czytamy od dołu: 1101_2

Konwersja z binarnego na dziesiętny – metoda potęg 2

Każdy bit przemnażamy przez odpowiednią potęgę 2, zależną od pozycji bitu. Suma tych iloczynów daje wynik w systemie dziesiętnym.

Przykład: 100101_2

$$1 \cdot 2^5 + 0 \cdot 2^4 + 0 \quad 2^3 + 1 \cdot 2^2 + 0 \quad 2^1 + 1 \cdot 2^0$$

$$= 32 + 0 + 0 + 4 + 0 + 1 = 37_{10}.$$

System oktalny i szesnastkowy

Oktalny (base 8):

- Cyfry: 0–7.
- **Przykład**:

$$177_8 = 1 \times 8^2 + 7 \times 8^1 + 7 \times 8^0$$
$$= 64 + 56 + 7 = 127_{10}.$$

Heksadecymalny (base 16):

- Cyfry: 0–9, A–F (A = 10, ..., F = 15).
- **Przykład**:

$$1A3_{16} = 1 \times 16^2 + 10 \times 16^1 + 3 \times 16^0$$
$$= 256 + 160 + 3 = 419_{10}.$$

System BCD

Idea
Każda cyfra dziesiętna jest kodowana osobno przy użyciu 4 bitów. Oznacza to, że każda cyfra liczby dziesiętnej jest zapisywana jako jej odpowiednik binarny w postaci 4-bitowej.
Przykład
Rozważmy liczbę dziesiętną 59:

- Cyfra 5 w systemie BCD: 0101
- Cyfra 9 w systemie BCD: 1001

Zatem liczba 59 w systemie BCD jest zapisana jako: 0101 1001.

$$59_{10} = 0101\ 1001_{BCD}$$

Porównanie z zapisem binarnym

Zwykła konwersja liczby 59 do systemu binarnego wygląda następująco:

$$59_{10} = 111011_2$$

Podczas gdy w systemie BCD każda cyfra jest zapisywana oddzielnie w 4-bitowych grupach, co daje:

$$59_{10} = 0101\ 1001_{BCD}$$

Wniosek: zapis BCD nie jest tym samym co zwykła reprezentacja binarna liczby!

Zastosowania systemu BCD

Liczba dziesiętna	Reprezentacja BCD
27	0010 0111
83	1000 0011
142	0001 0100 0010
205	0010 0000 0101

System BCD jest często stosowany w:

- Wyświetlaczach kalkulatorów – każda cyfra jest łatwa do odczytania i przetworzenia.
- Urządzeniach cyfrowych, takich jak zegarki elektroniczne, liczniki, sterowniki przemysłowe.
- Systemach wbudowanych, gdzie konwersja między liczbami dziesiętnymi a binarnymi musi być szybka i prosta.

Dlaczego kodowanie jest potrzebne?

Komputer przechowuje i przetwarza dane jako sekwencje 0 i 1 (bitów). Aby móc operować na różnych typach danych (tekst, liczby całkowite, liczby zmiennoprzecinkowe), trzeba je zakodować w formacie binarnym.

Kodowanie znaków

Aby komputer mógł przechowywać i przetwarzać litery, cyfry oraz symbole, opracowano standardy kodowania znaków. Najważniejsze standardy:

✔ **ASCII** (American Standard Code for Information Interchange)

Wykorzystuje 7 bitów (może kodować 128 znaków). Obejmuje litery (A-Z, a-z), cyfry (0-9) oraz podstawowe symbole (!, @, # itd.).

✔ **Unicode**

Rozszerza ASCII, obsługując wiele języków i symboli (np. ą, ę, ś, ć, emoji, chińskie znaki). Może wykorzystywać 8, 16 lub nawet 32 bity na znak.

Przykład kodowania ASCII:

Znak	Kod binarny	Kod dziesiętny
A	01000001	65
B	01000010	66
a	01100001	97

Unicode – przykłady znaków spoza ASCII:

Znak	Kod Unicode (szesnastkowy)
ą	U+0105
€	U+20AC

Reprezentacja znak-moduł (sign-magnitude)

Opis metody

Reprezentacja znak-moduł (sign-magnitude) to jeden z najprostszych sposobów kodowania liczb całkowitych w postaci binarnej.

Zasada działania:

Najbardziej znaczący bit (MSB – Most Significant Bit) oznacza znak liczby:

- 1 → liczba dodatnia
- 2 → liczba ujemna

Pozostałe bity przechowują wartość bezwzględną liczby.

Liczba dziesiętna	Reprezentacja binarna	Interpretacja
+5	00000101	MSB = 0 (liczba dodatnia), pozostałe bity to 5 w binarnym.
-5	10000101	MSB = 1 (liczba ujemna), pozostałe bity to 5, ale teraz interpretowane jako -5.

Reprezentacja uzupełnienia do jedynki (1’s complement)

Liczba dziesiętna	Kod binarny (1’s complement)
+5	00000101
-5	11111010

Opis metody

Reprezentacja uzupełnienia do jedynki (1's complement) to sposób zapisu liczb całkowitych w systemie binarnym, w którym liczby ujemne są tworzone poprzez odwrócenie (negację) wszystkich bitów liczby dodatniej.

Zasada działania:

Każdy 0 zamienia się na 1, a każdy 1 zamienia się na 0.

Problem: Dwie reprezentacje zera

Przykład: 8-bitowa reprezentacja liczby +5 i -5

Liczba dziesiętna	Kod binarny (1’s complement)
+5	00000101
-5	11111010

W systemie uzupełnienia do jedynki istnieją dwa sposoby zapisania zera:

+0 = 00000000

-0 = 11111111

To powoduje komplikacje w obliczeniach, ponieważ komputer musi dodatkowo obsługiwać oba przypadki.

Reprezentacja uzupełnienia do dwóch (2’s complement) – wprowadzenie

Opis metody

Reprezentacja uzupełnienia do dwóch (2’s complement) to najczęściej stosowany sposób zapisu liczb całkowitych w komputerach.

Jest szeroko używana w procesorach, ponieważ:

- upraszcza operacje arytmetyczne, eliminując problem podwójnego zera,
- umożliwia łatwe odejmowanie, ponieważ operacja odejmowania sprowadza się do dodawania liczby ujemnej.

Jak wyznaczyć liczbę -x w uzupełnieniu do dwóch?

Aby otrzymać liczbę -x, wykonujemy 3 kroki:

1. Zapisujemy liczbę dodatnią +x w postaci binarnej.
2. Negujemy wszystkie bity (zamieniamy 0 na 1 i 1 na 0).
3. Dodajemy 1 do wyniku.

Przykład: 8-bitowa reprezentacja liczby +8 i -8

Operacja	Wynik
Liczba +8 w systemie binarnym	00001000
Negacja wszystkich bitów	11110111
Dodanie 1	11111000
Ostateczna wartość (-8)	11111000

Zalety reprezentacji uzupełnienia do dwóch:

- Tylko jedno zero → 00000000, co eliminuje problem podwójnego zera obecny w uzupełnieniu do jedynki.
- Dodawanie i odejmowanie działa tak samo jak dla liczb bez znaku → upraszcza sprzętową implementację arytmetyki binarnej.
- Łatwe odejmowanie → operacja odejmowania sprowadza się do dodawania liczby ujemnej.

Wady reprezentacji uzupełnienia do dwóch:

- Asymetryczny zakres liczb – dla n-bitowego zapisu:
  - Najmniejsza liczba: −2^(n−1)
  - Największa liczba: 2^(n−1) - 1
- Przykład dla 8 bitów → zakres liczb: od -128 do +127
- Przepełnienie (overflow)

Konwersja między różnymi długościami bitowymi

Wprowadzenie

Czasami konieczne jest dostosowanie liczby binarnej do innej długości bitowej, np. gdy liczba 8-bitowa musi zostać zapisana w 16-bitowym rejestrze. W zależności od tego, czy liczba jest bez znaku czy zapisana w uzupełnieniu do dwóch, stosujemy różne zasady rozszerzania.

Rozszerzanie liczb bez znaku

Jeśli liczba jest bez znaku, dodajemy zera z lewej strony.

Przykład:

Liczba 8-bitowa: 00101100 (dziesiętnie 44)
Wersja 16-bitowa: 00000000 00101100
Wartość nie zmienia się.

Rozszerzanie liczb w uzupełnieniu do dwóch (2’s complement)

Przykład:

Jeśli liczba jest zapisana w uzupełnieniu do dwóch, powielamy bit znaku
(sign extension).

Liczba -5 w 8-bitowym zapisie:
11111011
Aby przekonwertować na 16 bitów, kopiujemy bit znaku (1) do nowych bitów z lewej strony:
11111111 11111011
Ostatecznie -5 w 16 bitach: 11111111 11111011 (wartość pozostaje bez zmian)

Dlaczego powielamy bit znaku?
W uzupełnieniu do dwóch powielanie bitu znaku pozwala zachować poprawną wartość liczby po zmianie długości bitów.

Porównanie konwersji 8-bitowych liczb do 16- bitowych

Liczba dziesiętna	8-bitowa reprezentacja	16-bitowa reprezentacja
+5	00000101	00000000 00000101
-5	11111011	11111111 11111011
+8	00001000	00000000 00001000
-8	11111000	11111111 11111000

Dodawanie liczb binarnych – 1- bitowe przypomnienie

Zasady dodawania binarnego
Dodawanie liczb binarnych działa podobnie do dodawania w systemie dziesiętnym, ale zamiast cyfr 0-9 operujemy tylko na 0 i 1. Kluczowym elementem jest przeniesienie, które wpływa na sumowanie kolejnych bitów.

Tablica prawdy dla dodawania 1-bitowego (bez przeniesienia)

Bit A	Bit B	Suma (Sum)	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1
Jeśli 1 + 1, suma na tej pozycji to 0, a przeniesienie (carry) przechodzi do następnego bitu w lewo.			

Przykład: 4-bitowe dodawanie z przeniesieniem

Dodajemy liczby 6 i 3:

```
  0110   (6 w dziesiętnym)
+ 0011   (3 w dziesiętnym)
----------
  1001   (9 w dziesiętnym)
```

Tutaj nie ma przeniesienia poza zakres 4 bitów, więc wynik jest poprawny.

Dodawanie w uzupełnieniu do dwóch – przykład

Zasady działania
Dodawanie liczb ujemnych w uzupełnieniu do dwóch działa identycznie jak dodawanie liczb dodatnich, ponieważ liczby ujemne są już zapisane w poprawnym formacie binarnym.

Bit A	Bit B	Wynik sumy
0	0	0
0	1	1
1	0	1
1	1	0 (z przeniesieniem 1)

Przykład: Dodawanie +5 i -3 (8-bitowy zapis)

1. Zapisujemy +5 w postaci binarnej (8-bitowej):

00000101 (+5)

2. Zapisujemy -3 w uzupełnieniu do dwóch:

+3 w binarnym: 00000011

Negacja bitów: 11111100

Dodanie 1:

111111001

11111101 (-3 w uzupełnieniu do dwóch)

3. Dodajemy liczby:

```
  00000101 (+5)
+ 11111101 (-3)
----------------
  00000010 (Wynik: +2)
```

Interpretacja wyniku: Wynik 00000010 to +2, co zgadza się z oczekiwanym wynikiem: 5 + (-3) = 2

Przeniesienie z 9. bitu jest ignorowane, ponieważ działamy na 8-bitowych liczbach.

Reguła przepełnienia (overflow)

Co to jest przepełnienie?
Przepełnienie występuje, gdy wynik nie mieści się w zakresie reprezentacji dostępnej dla danej liczby bitów. Dotyczy zarówno liczb ze znakiem (w uzupełnieniu do dwóch), jak i bez znaku.

Zasada wykrywania przepełnienia
Przepełnienie może wystąpić, gdy:

- Dodajemy dwie liczby o tym samym znaku, a wynik wychodzi z przeciwnym znakiem.
- Odejmujemy dwie liczby o przeciwnych znakach, a wynik przekracza zakres.

W systemie uzupełnienia do dwóch (2’s complement) przepełnienie wykrywa się na podstawie bitu znaku:

- Dwie liczby dodatnie → wynik ujemny → przepełnienie.
- Dwie liczby ujemne → wynik dodatni → przepełnienie.

Przykład: Dodawanie +120 i +50 w systemie uzupełnienia do dwóch
Zapis dziesiętnych wartości w postaci binarnej (8 bitów)

```
  01111000 (120 )
+ 00110010 (50)
------------------
10001010  (w 8 bitach = -118)
```

Interpretacja wyniku:
Otrzymaliśmy 10001010, co w uzupełnieniu do dwóch oznacza -118.
Oczekiwany wynik powinien być dodatni (170), ale uzyskaliśmy liczbę ujemną.
To oznacza przepełnienie!

Dodatkowy przykład przepełnienia przy odejmowaniu

Rozważmy: -100 - 50 w 8-bitowym systemie uzupełnienia do dwóch

Zapis binarny liczb

```
   10011100  (-100)
 - 11001110  (-50)
  ------------------
    01101010  (w 8 bitach = +106)
```

Interpretacja wyniku:

Otrzymaliśmy +106, ale wynik powinien być ujemny.

To także przepełnienie!

Podsumowanie wykrywania przepełnienia

- Przepełnienie występuje, gdy bit znaku wyniku różni się od bitów znaku operandów.
- Dodanie dwóch liczb dodatnich dało liczbę ujemną → przepełnienie.
- Dodanie dwóch liczb ujemnych dało liczbę dodatnią → przepełnienie.
- Przepełnienia nie wykrywa się na podstawie przeniesienia bitowego, ale na podstawie zmiany bitu znaku.

Mnożenie – liczby bez znaku (metoda sumowania i przesuwania)

Algorytm mnożenia binarnego
Algorytm mnożenia binarnego działa na zasadzie sumowania i przesuwania. Działa analogicznie do mnożenia pisemnego, ale zamiast mnożyć, przesuwamy bity i dodajemy wartości.
Kroki algorytmu (dla uproszczonego przykładu 4-bitowego):

Sprawdź najmłodszy bit mnożnika (LSB – Least Significant Bit). Jeśli jest 1, dodaj wartość mnożnej do wyniku.

Przesuń mnożną w lewo (czyli pomnóż ją przez 2) i mnożnik w prawo (podziel przez 2).

Powtarzaj kroki dla każdego bitu mnożnika.

Przykład: Mnożenie 6 × 5 w systemie binarnym

Dane wejściowe:
6 w binarnym: 0110
5 w binarnym: 0101
Kroki obliczeniowe:

Krok	Mnożna (6)	Mnożnik (5)	Wynik (sumowanie)
1	0110	0101	+ 0110 (bo LSB = 1)
2	1100 (×2)	0010 (/2)	brak zmiany
3	11000 (×2)	0001 (/2)	+ 11000 (bo LSB = 1)
4	00000 (×2)	0000 (/2)	brak zmiany

Ostateczna suma:

0110

+11000

11110 (czyli 30 w dziesiętnym)

Uwagi do metody

✔ Każdy bit mnożnika decyduje, czy mnożna zostanie dodana do wyniku.

✔ Przesunięcia binarne pozwalają na szybkie obliczenia zamiast tradycyjnego mnożenia.

✔ W procesorach metoda ta jest wykorzystywana w jednostkach ALU do implementacji mnożenia sprzętowego.

Mnożenie w uzupełnieniu do dwóch – uwaga o algorytmie Bootha

Uwzględnienie bitu znaku w mnożeniu
Podczas mnożenia liczb całkowitych zapisanych w uzupełnieniu do dwóch (2’s complement) konieczne jest uwzględnienie bitu znaku. Liczby ujemne i dodatnie są kodowane w specjalny sposób, dlatego proste mnożenie bitowe może prowadzić do błędów, jeśli nie zostanie odpowiednio obsłużone.

Algorytm Bootha
Algorytm Bootha jest często stosowany do optymalnego mnożenia liczb w uzupełnieniu do dwóch. Główne zalety algorytmu Bootha:

- Umożliwia efektywne mnożenie liczb ujemnych i dodatnich.
- Wykorzystuje operacje przesunięć i dodawania/odejmowania, co upraszcza sprzętową realizację w procesorach.
- Pozwala na redukcję liczby operacji, co wpływa na wydajność.
- W praktyce procesory i jednostki arytmetyczno-logiczne (ALU) realizują ten algorytm automatycznie.

Metoda ręczna mnożenia liczb w uzupełnieniu do dwóch

Jeśli wykonujemy mnożenie ręcznie, możemy postępować według uproszczonej reguły:

1. Określenie znaku wyniku
   - (-) × (+) lub (+) × (-) → wynik ujemny
   - (-) × (-) lub (+) × (+) → wynik dodatni
2. Mnożenie wartości bezwzględnych
3. Ignorujemy na chwilę bit znaku i wykonujemy standardowe mnożenie binarne.

Zapisanie znaku w uzupełnieniu do dwóch Jeśli wynik powinien być ujemny, konwertujemy go do formatu 2’s complement (uzupełnienia do dwóch).

Przykład: Mnożenie (-6) × (+5) w 8-bitowym uzupełnieniu do dwóch

Reprezentacja liczb w uzupełnieniu do dwóch (8 bitów):

-6 = 11111010

+5 = 00000101

Mnożymy wartości bezwzględne (6 × 5 = 30):

6 = 00000110

5 = 00000101

Wynik mnożenia (bez znaku) = 00011110 (30 dziesiętnie)

Konwersja na liczbę ujemną w uzupełnieniu do dwóch:

1. Negujemy bity:
2. 00011110 → 11100001
3. Dodajemy 1:
4. 11100001 + 1 = 11100010

Ostateczny wynik: 11100010 (-30 w 8-bitowym uzupełnieniu do dwóch)

Wnioski:

- Algorytm Bootha jest szeroko stosowany do optymalizacji mnożenia liczb w systemie 2’s complement.
- Przy ręcznym mnożeniu należy uwzględnić bit znaku, co wymaga dodatkowej operacji po zakończeniu mnożenia wartości bezwzględnych.
- Procesory realizują mnożenie w 2’s complement automatycznie, bez konieczności ręcznej konwersji znaku.

Dzielenie – liczby bez znaku (metoda odejmowania i przesuwania)

Zasada działania
Dzielenie binarne opiera się na metodzie odejmowania i przesuwania, w której:

W każdej iteracji sprawdzamy, czy można odjąć dzielnik od fragmentu dzielnej.

Jeśli tak, odejmujemy i zwiększamy wynik.

Przesuwamy bity, aby sprawdzić kolejne pozycje.

Kroki algorytmu dzielenia binarnego:

Ustaw wynik na 0.

Porównaj dzielnik z częścią dzielnej.

Jeśli można odjąć, wykonaj odejmowanie i zwiększ wynik.

Przesuń dzielnik w prawo, powtarzaj operację aż do końca bitów.

Przykład: Dzielenie 13 ÷ 3 w systemie binarnym

Zapis dzielnej i dzielnika w postaci binarnej:

13 (dzielna) = 1101

3 (dzielnik) = 0011

Przebieg operacji (odejmowanie i przesunięcia):

Iteracja	Część dzielnej	Można odjąć?	Wynik (bit)	Nowy stan
1	11	Tak	1	11 - 11 = 00
2	00 (przesuwamy)	Nie	0	00
3	01	Nie	0	01
4	1101 (przesunięta dzielna)	Tak	1	1101 - 0011 = 1010

Ostateczny wynik: 13 ÷ 3 w systemie binarnym daje wynik 0101 (czyli 4 dziesiętnie) z resztą 1.

Uwagi do metody

✓ Każdy bit dzielnej jest przetwarzany pojedynczo, co pozwala na precyzyjne wyznaczanie ilorazu.

✓ Wykorzystanie odejmowania i przesunięć eliminuje konieczność wykonywania tradycyjnego dzielenia.

✓ Metoda ta jest stosowana w sprzętowych jednostkach dzielących procesorów.

Dzielenie w uzupełnieniu do dwóch (2’s complement)

Zasada działania

Dzielenie liczb zapisanych w uzupełnieniu do dwóch (2’s complement) działa podobnie jak dla liczb bez znaku, ale wymaga dodatkowej obsługi bitu znaku (±).

Kroki algorytmu dzielenia w 2’s complement:

1. Sprawdź znaki dzielnej i dzielnika. Jeśli oba mają ten sam znak, wynik będzie dodatni. Jeśli znaki są różne, wynik będzie ujemny.
2. Dziel wartości bezwzględne (ignorując na chwilę znak).
3. Jeśli wynik ma być ujemny, zamień go na postać uzupełnienia do dwóch.

Przykład: Dzielenie -14 ÷ +2

Zapis liczb w systemie binarnym (8 bitów, 2’s complement):

-14 = 11110010

+2 = 00000010

Dzielimy wartości bezwzględne:

14 ÷ 2 = 7

7 w binarnym: 00000111

Sprawdzamy znak wyniku:

- -14 ma 1 (znak ujemny).
- +2 ma 0 (znak dodatni).

Znaki są różne → wynik będzie ujemny.

Konwersja 7 na -7 w uzupełnieniu do dwóch:

1. Negacja bitów:
2. 00000111 → 11111000
3. Dodanie 1:
4. 11111000 + 1 = 11111001

Ostateczny wynik: 11111001 (-7 w 8-bitowym uzupełnieniu do dwóch).

Uwagi do metody

✔ Obsługa bitu znaku pozwala na poprawne wykonywanie dzielenia bez konieczności oddzielnej logiki dla liczb ujemnych.

✔ Po wykonaniu operacji dzielenia, konwersja znaku wyniku odbywa się automatycznie poprzez zamianę na 2’s complement, jeśli znaki operandów były różne.

✔ Procesory wykorzystują ten mechanizm sprzętowo, co eliminuje konieczność manualnej obsługi bitu znaku.

Reprezentacja zmiennoprzecinkowa – wprowadzenie

Cel
Reprezentacja zmiennoprzecinkowa pozwala na zapisywanie bardzo dużych oraz bardzo małych liczb w ograniczonej liczbie bitów.
Jest niezbędna w obliczeniach naukowych, inżynierskich i finansowych.

Standard IEEE 754

IEEE 754 to standard zapisu liczb zmiennoprzecinkowych w systemach komputerowych.
Dwa podstawowe formaty:

- Single Precision (32 bity)
- Double Precision (64 bity)

Każda liczba zmiennoprzecinkowa jest przedstawiana według wzoru:

$$(-1)^{\text{znak}} \times 1.\text{mantysa} \times 2^{(\text{cecha}-\text{bias})}$$

Składniki reprezentacji

Dlaczego zmiennoprzecinkowa?

- Pozwala na zapis liczb w szerokim zakresie (np. 1.23 × 10^30 i 1.23 × 10^–30) w tej samej liczbie bitów.
- Jest powszechnie stosowana w procesorach, kartach graficznych i systemach obliczeniowych.
- Umożliwia szybkie operacje na liczbach rzeczywistych dzięki sprzętowej implementacji w FPU (Floating Point Unit).

Składnik	Opis
Bit znaku	Określa, czy liczba jest dodatnia (0) czy ujemna (1).
Mantysa (significand)	Zawiera znaczące cyfry liczby w postaci znormalizowanej.
Cecha (exponent)	Określa przesunięcie dziesiętne (potęga liczby 2).
Bias	Stała wartość odejmowana od zapisanej cechy, pozwalająca przechowywać dodatnie i ujemne wykładniki.

Struktura
Format Single Precision (32-bitowy) w standardzie IEEE 754 składa się z:

Składnik	Opis
1 bit znaku	Określa, czy liczba jest dodatnia (0) czy ujemna (1).
8 bitów cechy (exponent)	Koduje potęgę liczby 2, z przesunięciem (bias) = 127.
23 bity mantysy	Przechowują znaczące cyfry liczby w postaci znormalizowanej.

Przykład: Reprezentacja liczby 3,75 w formacie Single Precision

1. Zapis liczby w systemie binarnym
2. Liczba 3,75 w systemie binarnym:
3. $3{,}75_{10} = 11.11_2$
4. W postaci znormalizowanej:
5. 1.111×2^1
6. Mantysa przechowuje tylko część po przecinku:
7. 11100000000000000000000 (uzupełniona zerami do 23 bitów)
8. Obliczenie wartości cechy
9. Wykładnik wynosi 1
10. Bias wynosi 127
11. Zapisujemy:
12. 1 + 127 = 128
13. W postaci binarnej:
14. $128_{10} = 10000000_2$

Złożenie całości

Bit	Wartość
Bit znaku	0 (bo liczba jest dodatnia)
Cecha	10000000
Mantysa	11100000000000000000000

Ostateczny zapis 3,75 w IEEE 754 (Single Precision, 32-bit):

0 | 10000000 | 11100000000000000000000

Uwagi dotyczące Single Precision

✔ Single Precision umożliwia zapis liczb w zakresie:

$$\pm 1.18 \times 10^{-38} \quad \text{do} \quad \pm 3.4 \times 10^{38}$$

✔ Ograniczona precyzja (tylko 23 bity mantysy) sprawia, że niektóre liczby nie mogą być reprezentowane dokładnie, co może prowadzić do błędów zaokrągleń.
✔ Bias 127 pozwala na reprezentację zarówno liczb dodatnich, jak i ujemnych wykładników.

Format Double Precision (64-bit)

Liczby zmiennoprzecinkowe w formacie Double Precision
według standardu IEEE 754 składają się z:

Składnik	Opis
1 bit znaku	Określa, czy liczba jest dodatnia (0) czy ujemna (1).
11 bitów cechy (exponent)	Koduje wykładnik potęgi 2, z biasem 1023.
52 bity mantysy	Przechowują znaczące cyfry liczby w postaci znormalizowanej.

Cechy Double Precision:

Większy zakres wartości w porównaniu do Single Precision, co pozwala na przechowywanie ekstremalnie małych i dużych liczb.
Lepsza precyzja – 52 bity mantysy pozwalają na dokładniejsze reprezentowanie liczb.
Bias równy 1023 umożliwia obsługę dodatnich i ujemnych wykładników.

Zakres wartości Double Precision:
Zakres wartości liczbowych: $\pm 2.23 \times 10^{-308}$ do $\pm 1.8 \times 10^{308}$
Możliwość przechowywania większej ilości cyfr znaczących w obliczeniach numerycznych.
Stosowany w naukowych obliczeniach, modelach fizycznych i inżynierskich wymagających dużej precyzji.

Arytmetyka zmiennoprzecinkowa – dodawanie i odejmowanie

Etapy dodawania i odejmowania liczb zmiennoprzecinkowych
Podczas dodawania i odejmowania liczb zmiennoprzecinkowych kluczowe jest wyrównanie cech (eksponentów).
Proces składa się z kilku etapów:

Wyrównanie cech (eksponentów)
Przed wykonaniem operacji należy dostosować wykładniki potęgi 2, aby obie liczby miały tę samą podstawę.

Sumowanie lub odejmowanie mantys

Po wyrównaniu cech dodajemy lub odejmujemy mantysy.

Normalizacja wyniku

Jeśli mantysa wyniku nie mieści się w przedziale [1, 2), konieczna jest ponowna normalizacja poprzez przesunięcie bitów i dostosowanie cechy.

Zaokrąglenie (rounding)
Reprezentacja IEEE 754 ma ograniczoną precyzję, więc wynik może wymagać zaokrąglenia (do najbliższej wartości, w górę, w dół, do zera itp.).

Przykład dodawania zmiennoprzecinkowego

Dodajemy dwie liczby zapisane w systemie zmiennoprzecinkowym:
$1{,}101_2 \times 2^3 + 1{,}001_2 \times 2^2$
Wyrównanie cech
Mniejszą cechę (2^2) zwiększamy do 2^3, przesuwając mantysę w prawo:
$1{,}001_2 \times 2^2 = 0{,}1001_2 \times 2^3$
Dodawanie mantys
Dodajemy mantysy:

$$
\begin{array}{r}
1{,}101_2 \\
+\ 0{,}1001_2 \\
\hline
10{,}0011_2
\end{array}
$$

Normalizacja wyniku
Wynik $10{,}0011_2 \times 2^3$ nie jest w postaci znormalizowanej ($1.xxxx \times 2^n$).
Przesuwamy mantysę w prawo i zwiększamy cechę o 1:
$1{,}00011_2 \times 2^4$
Ostateczny wynik:
$1{,}00011_2 \times 2^4$

Uwagi dotyczące arytmetyki zmiennoprzecinkowej:
✔ Operacje na liczbach zmiennoprzecinkowych są wolniejsze niż operacje na liczbach całkowitych, ponieważ wymagają wyrównywania cech i normalizacji.
✔ Zaokrąglenie może powodować utratę precyzji, co jest istotne w obliczeniach numerycznych.
✔ Sprzętowa implementacja operacji zmiennoprzecinkowych w procesorach (FPU) pozwala na szybkie obliczenia matematyczne w standardzie IEEE 754.